

Topic: Stack using Singly Linked List

Q. Explain how a Stack can be implemented using a Singly Linked List. Provide the logic for the Push and Pop operations.

[DEF] Definition to Remember

A Stack follows the LIFO (Last In, First Out) principle. Implementing it using a Singly Linked List allows the stack to grow dynamically, preventing Stack Overflow (unless heap memory runs out). The Head of the linked list acts as the Top of the stack.

Stack Implementation using Singly Linked List

To maintain $O(1)$ time complexity for both Push and Pop operations, all insertions and deletions must occur at the beginning (head) of the linked list.

Stack Operation	Linked List Equivalent
Top Pointer	`head` pointer
Push(value)	Insert node at the **beginning**
Pop()	Delete node from the **beginning**

2. Push Operation (Insert at Beginning)

Adds a new item to the top of the stack.

Algorithm:

```
1. All
2. If
3. new
4. new
5. top
```

3. Pop Operation (Delete from Beginning)

Removes and returns the item from the top of the stack.

Algorithm:

```
1. If
2. tem
3. val
4. top
5. fre
6. ret
```

4. Advantages of Linked List Implementation

* **Dynamic Size:** The stack can grow and shrink dynamically. It is limited only by system memory, unlike an array which has a fixed pre-defined size.

* No

[MUST-WRITE] Must-Write Points (for 10 marks)

- Stack is LIFO. Linked List implementation makes the stack dynamic in size.
- Prevents "Stack Overflow" related to fixed array limits (only fails if heap memory is exhausted).
- The Head of the linked list represents the Top of the stack.
- Push = Insert at Beginning. Time complexity is $O(1)$.
- Pop = Delete from Beginning. Time complexity is $O(1)$.
- Push Logic: `new_node->next = top; top = new_node;`

7. Pop Logic: `temp = top; top = top->next; free(temp);`

[RECALL] Quick Recall

Stack via Linked List -> Dynamic Size (no array limits) -> Head = Top -> Push = Insert at Head ($O(1)$) -> Pop = Delete from Head ($O(1)$)